# Sorting in linear time

*Part II Sorting and Order Statistics*

## Scope and prerequisites

Linear sorting exchanges restrictions on key representations or input distributions for speed. Distinguish the comparison-model lower bound, then derive counting-sort blocks, LSD stability, and squared bucket-size costs. State ranges, bit lengths, distributions, and space requirements. Prerequisites are comparison trees, integer ranges, and stable sorting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chapter route

- Prove the Ω(n log n) comparison-sorting lower bound.
- Analyze counting-sort assumptions and stability.
- Identify the cost premises of radix and bucket sort.

Sections 8.1–8.4 cover lower bounds, counting sort, radix sort, and bucket sort.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

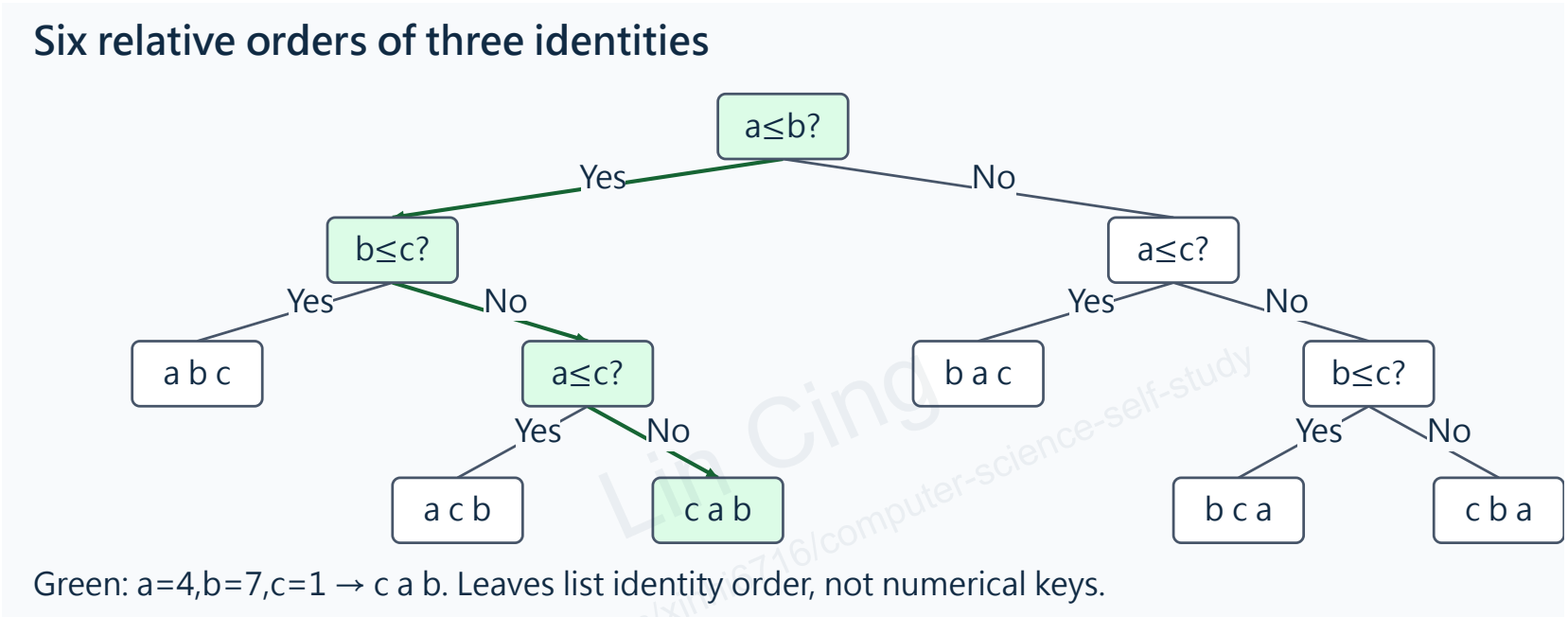
## 8.1 Lower bounds for sorting

A comparison sort obtains order information only through key comparisons, not by using keys as indices. Restrict first to n distinct keys, making each $\leq$ comparison binary. For a fixed n and algorithm, an internal node names two original identities to compare, a root-to-leaf path records an execution, and a leaf determines the entire identity order.

The n! relative orders need distinguishable leaves. If two different orders reached the same leaf, the same result sequence would force the same output identity order, making at least one input wrong. Several leaves may represent the same output, but at least n! reachable leaves are necessary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 8.1 Lower bounds for sorting (continued)




A leaf such as “c a b” means c<a<b, not six arbitrary numerical outputs. The input a=4,b=7,c=1 takes left,right,right to reach c a b.

# Quantifiers and model of the worst-case bound

A binary tree of height h has at most $2^h$ leaves, so $h \geq \text{ceil}(\log_2(n!))$. Without Stirling, $n! \leq n^n$, while its final ceil(n/2) factors are each at least n/2. Hence

$$(n/2)^{n/2} \leq n! \leq n^n, \qquad \log_2(n!) = \Theta(n \log n).$$

For every correct comparison sort and each n, **some** length-n input therefore requires Ω(n log n) comparisons. Not every input does: sorted insertion-sort input uses n−1 comparisons.

Distinct keys form a hard subset even when duplicate inputs are allowed, so allowing duplicates does not remove the general worst-case bound. Counting and radix sorts read integer representations and index arrays, leaving the comparison model. They incur range and bit-length costs instead of contradicting the theorem.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Average comparisons require a proof too

Suppose all N=n! orders are uniform. Choose a reachable leaf for each, with depths $d_1,\ldots,d_N$. Distinct leaf paths form prefix-free binary strings. Kraft's inequality $\Sigma 2^{(-d_i)} \leq 1$ follows by assigning each depth-d leaf a disjoint binary interval of length $2^{(-d)}$ in [0,1).

Arithmetic mean at least geometric mean gives

$$2^{-\sum_i d_i/N} \leq \frac{\sum_i 2^{-d_i}}{N} \leq 1/N.$$

Taking logs proves average depth at least $\log_2 N = \Omega(n \log n)$. A deepest-leaf bound alone would not prove this average.

For an always-correct randomized comparison sort, fixing random bits gives a correct tree. Its uniform-input average obeys the same bound. Averaging over random bits preserves it, so some input also has expected comparisons $\Omega(n \log n)$. This argument does not cover error-permitting Monte Carlo sorting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 8.2 Counting sort

Keys are integers 0..k. Satellite data travels with its record. Input/output positions are 1-based; count-table keys are 0-based.

```
C[0..k]=0
for j=1..n: C[key(A[j])]++
for v=1..k: C[v]+=C[v-1]
for j=n downto 1:
    v=key(A[j])
    B[C[v]]=A[j]
    C[v]--
return B
```

After counting, C[v] is key v's frequency. After prefix summing, it is the number of records with keys $\leq$v, the final position of v's block. Placement consumes C. Save the original prefix table P first if range queries are needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

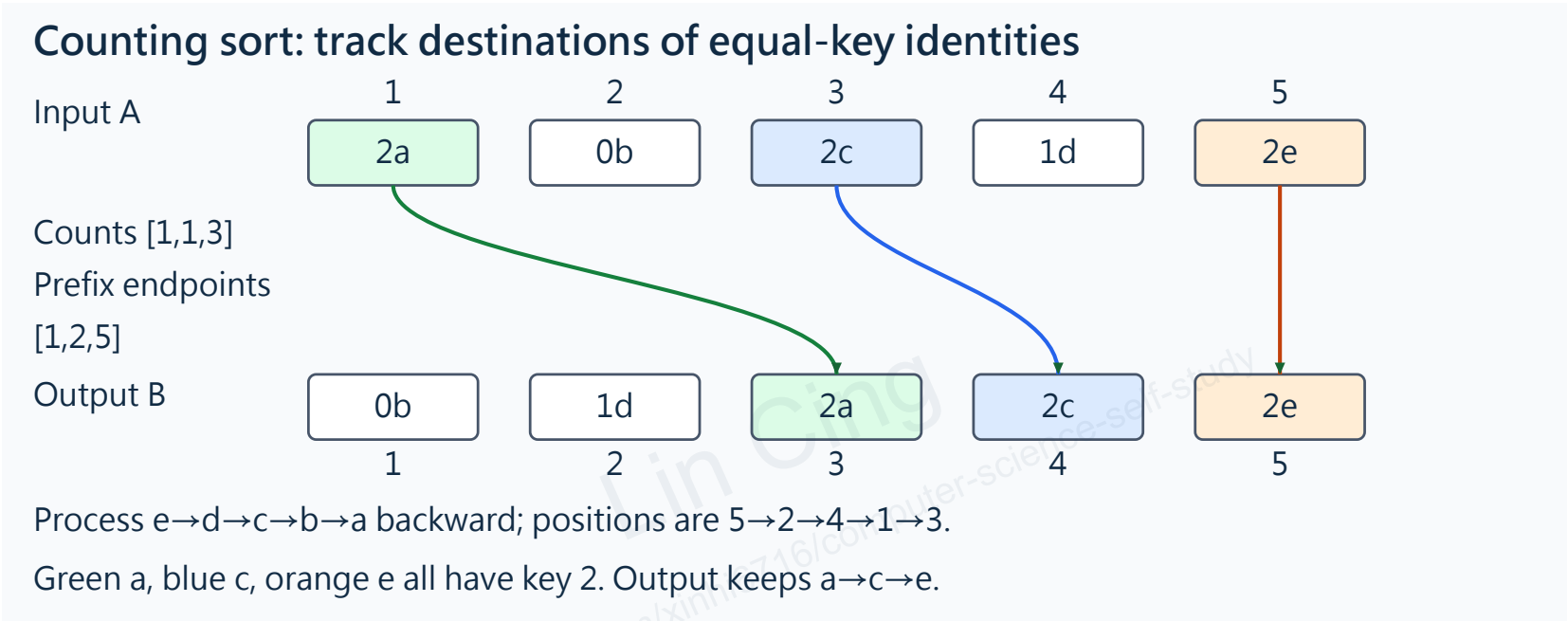
# From frequencies to output positions

Use A=[2a,0b,2c,1d,2e], with identities a,b,c,d,e. Frequencies are [1,1,3], cumulative block endpoints [1,2,5]. Key 0 occupies position 1; key 1 occupies 2; key 2 occupies 3..5.

| j | Record | Output position | Updated C |
|---|---|---|---|
| 5 | 2e | 5 | [1,2,4] |
| 4 | 1d | 2 | [1,1,4] |
| 3 | 2c | 4 | [1,1,3] |
| 2 | 0b | 1 | [0,1,3] |
| 1 | 2a | 3 | [0,1,2] |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# From frequencies to output positions (continued)




Output is [0b,1d,2a,2c,2e]. Stability requires checking a,c,e, not just bare keys.

## Placement invariant, stability, and costs

For key v, set L=P[v−1], or zero at v=0, and R=P[v]. After processing a suffix, its v-records occupy the right end of [L+1,R]. C[v] points to that block's rightmost unfilled position. The next backward-scanned v fills C[v], then decrements it, without duplication or leaving the block. Finally every block contains exactly its frequency, with blocks ordered by key, proving correctness.

If x precedes y with the same key, backward scanning puts y farther right, then x farther left, preserving their order. Forward scanning with the same decrementing endpoints reverses equal identities. Forward scanning can instead be stable with incrementing **starting** positions; do not mix these specifications.

Time and extra space are $\Theta(n+k+1)$, including output B, so this is not in-place. Negative keys can shift by v−min, but range k=max−min may remain huge. Reading whole keys and indexing are unit-cost word-RAM assumptions.

Saved P answers a closed range $0 \le a \le b \le k$ by P[b]−P[a−1], with P[−1]=0. For [1,2], the example gives P[2]−P[0]=5−1=4 records. Queries cost $O(1)$ after $\Theta(n+k+1)$ preprocessing.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 8.3 Radix sort

For fixed d-digit nonnegative integers with each digit 0..r−1, LSD radix sort processes digits from least to most significant, using a stable sort each pass.

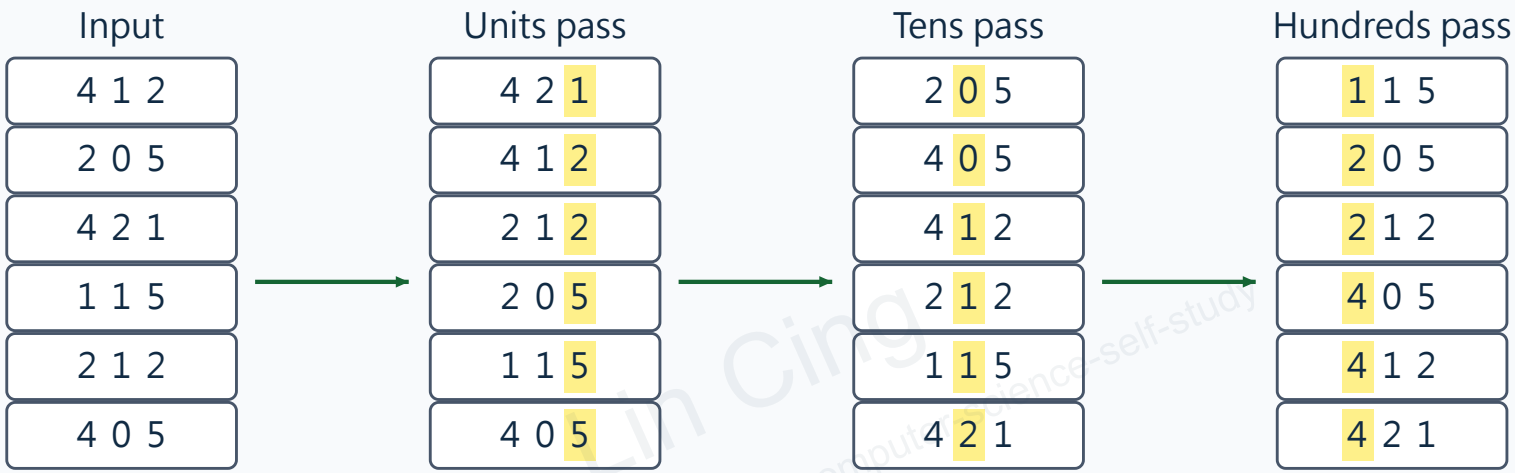
After t passes the sequence is sorted by its lowest t digits. The t=0 condition is empty. Pass t groups by that digit; different groups have the correct order, while stability within each group preserves the previous lower-digit order. This proves the invariant and complete sorting at t=d.

Lower-digit order is retained **within a common higher-digit group**, not globally. Unstable digit sorting breaks the induction: [12,11] becomes [11,12] after units, but a tie reversal on tens yields the incorrect [12,11].

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Three complete radix passes

## LSD radix: lower digits break higher-digit ties




Yellow marks the current digit. Equal current digits retain the previous column order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three complete radix passes (continued)

| Stage | Entire sequence |
|---|---|
| Input | [412,205,421,115,212,405] |
| Units | [421,412,212,205,115,405] |
| Tens | [205,405,412,212,115,421] |
| Hundreds | [115,205,212,405,412,421] |

In the tens pass, 205,405 both have digit 0 and retain that order. Records 412,212,115 all have tens digit 1 and retain their previous order. In the hundreds pass, 205 precedes 212 because the lower pair 05<12 is already ordered.

Sorting the entire array hundreds→tens→units, even stably, can make units the final primary key. Correct MSD sorting instead recurses within equal higher-digit groups; it is a different algorithm.

# Bit length, radix, and the cost model

Stable counting costs $\Theta(n+r)$ per digit, giving $\Theta(d(n+r))$ time and $\Theta(n+r)$ extra space. Constant $d$ and $r=O(n)$ directly give $\Theta(n)$. Long keys requiring nonconstant digit extraction need additional access costs.

For $b$-bit keys with $s$ bits per pass, $1 \le s \le b$, $r=2^s$ and $d=\text{ceil}(b/s)$. Cost is $\Theta(\text{ceil}(b/s)(n+2^s))$. Zero-pad a shorter highest digit. For $n \ge 2$ choose $s=\min(b,\text{floor}(\log_2 n))$, so $2^s \le n$ and time is $O(n\ \text{ceil}(b/\min(b,\text{floor}(\log_2 n))))$. If $b=O(\log n)$, this is $O(n)$.

For 32-bit keys in the simplified equal-unit model, $s=8$ costs $4(n+256)$, while $s=16$ costs $2(n+65536)$. The latter wins iff $n>65024$, with equality at 65024. These are modeled coefficients, not a measured crossover; cache, allocation, and bandwidth can change implementation choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 8.4 Bucket sort

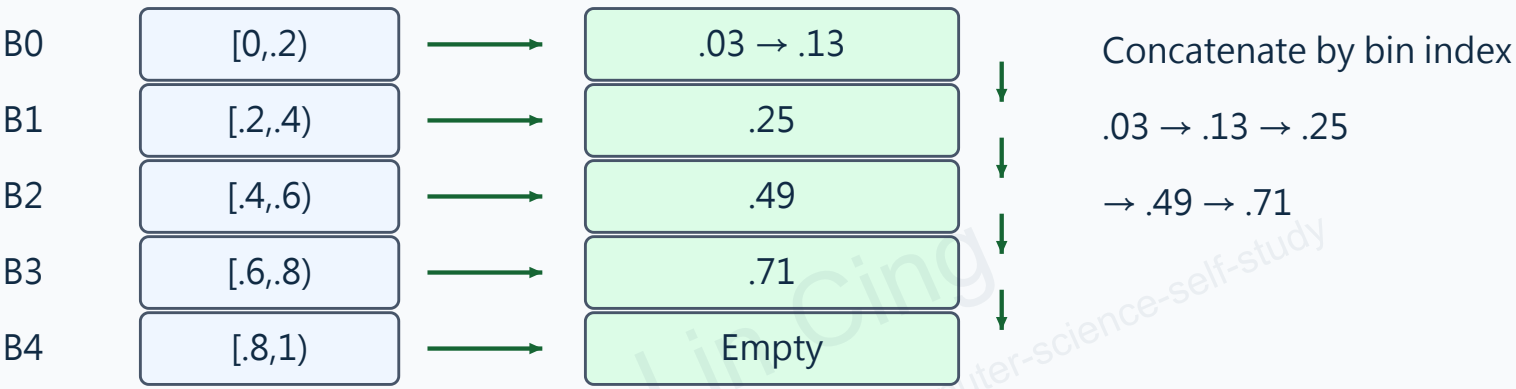
For $n \geq 1$ real values $x$ in $[0,1)$, create $n$ bins $B[0..n-1]$, append $x$ to $B[\lfloor nx \rfloor]$, insertion-sort each bin, and concatenate in index order. Empty input returns immediately. The value 1 is excluded; it would address nonexistent $B[n]$.

Correctness needs no distribution: $x \leq y$ implies $\lfloor nx \rfloor \leq \lfloor ny \rfloor$. Distinct bins are already ordered by their intervals, and sorting within bins completes the order. Stability additionally requires input-order append and stable insertion sort; head insertion would first reverse equal records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Bucket intervals guarantee cross-bin order

Input .13,.25,.03,.71,.49 → bins 0,1,0,3,2




Blue: half-open bin intervals. Green: sorted contents. No further cross-bin comparisons.

For [.13,.25,.03,.71,.49], indices are [0,1,0,3,2], not [0,0,0,1,2]. Sorted bins are $B_0$=[.03,.13],$B_1$=[.25],$B_2$=[.49],$B_3$=[.71],$B_4$ empty, giving [.03,.13,.25,.49,.71].

# Linear expectation requires squared bin sizes

Let $N_i$ be bin size. Initialization, distribution, and concatenation cost $\Theta(n)$; internal insertion sorts total $O(\Sigma N_i^2)$. Thus $T=O(n+\Sigma N_i^2)$. Knowing $E[N_i]=1$ does not make $E[N_i^2]=1$.

Assume independent uniform inputs in $[0,1)$. Let $X_{ij}$ indicate record $j$ in bin $i$. Then $N_i=\Sigma_j X_{ij}$, $X_{ij}^2=X_{ij}$, and $E[X_{ij}]=1/n$. Independence of distinct records gives $E[X_{ij}X_{ik}]=1/n^2$ for $j\neq k$. Consequently

$$E[N_i^2] = \sum_j 1/n + \sum_{j\neq k} 1/n^2 = 2 - 1/n.$$

The sum has expectation $2n-1$, giving $O(n)$; reading $n$ records gives $\Omega(n)$, hence $\Theta(n)$. Different bin sizes are dependent because their sum is $n$. Their independence is unnecessary for linearity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Distribution, dependence, and worst cases

For independent records with bin probabilities $p_i$ summing to 1, the same expansion gives

$$E[\sum_i N_i^2] = n + n(n-1)\sum_i p_i^2.$$

Uniform probabilities give sum of squares 1/n. More generally O(1/n) suffices for linear expected work. If one bin has probability 1, squared size is $n^2$. Concentration increases pair-collision probability $\Sigma p_i^2$.

Uniform marginals alone fail: choose J uniformly from 0..n−1 and set every record to (J+1/2)/n. Each record has uniform bin marginals and $E[N_i]=1$, but every execution has $\Sigma N_i^2=n^2$. This disproves the squared-work inference, not a claim that equal keys make stable insertion sort quadratic.

An actual worst-case family is $x_j = (n-j+1)/(2n^2)$,j=1..n,n≥2. All values occupy the first bin in strictly decreasing order, forcing n(n−1)/2 shifts and $\Theta(n^2)$ time. Replacing internal sorting with worst-case O(m log(m+1)) yields total worst-case O(n log n) while preserving linear expectation under independent uniform inputs, since m log(m+1)=$O(m^2)$.

# Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study